Counterglass: Stateful Counterfactual Failure Evidence and a Budgeted Search Study

Nazeeh Abdul-Hadi

16 October 2026

Abstract

An agent's successful response does not establish that its tool actions produced the required state. Counterfactual tests can expose this gap, but their conclusions depend on valid interventions, faithful restoration, independent outcome checks and the cost of reproducing a failure. We present COUNTERGLASS, a bounded testing system that checkpoints actual SQLite and file state together with a resumable agent state, separates committed tool effects from delivered observations, and emits independently checkable failure evidence. We study a simple certificate-completion index that scores the predictive probability of finishing a fixed confirmation obligation per expected testing cost, with and without an empirical prior pooled over public tool features. A locked study evaluates 10 policies over 32 seeds and three authored workflow structures. At the primary additional budget of 480 operation units, the pooled index finds 4.031 requirement/intervention signatures versus 3.812 for the independent index; the paired mean difference is +0.219 with descriptive 95% seed-bootstrap interval [+0.094, +0.375]. The frozen composite improvement target is not met. This is a negative result for the tested selection hypothesis at the prespecified criterion. An upstream ToolSandbox integration and a small local-model case study provide separate execution evidence, not selector-generalization results. The contribution is inspectable stateful testing infrastructure and a transparent empirical study; algorithmic originality and broad discovery superiority are not established.

1 Introduction

Consider a tool that commits a requested database write and then loses its acknowledgment. The agent observes a timeout, retries, and reports completion. Every eventual tool response can look reasonable while the database contains two copies of the requested job. A test that inspects only the final response, exit status or acknowledgment misses the affected requirement: exactly one committed operation.

Environmental counterfactuals offer a direct way to exercise such behavior. Starting from an otherwise successful task, restore a precise execution boundary, apply a permitted change, continue the agent and independently inspect the resulting state. The change might affect an observation, an external transition or an available permission. These are different interventions. Suppressing an acknowledgment must preserve the write; replacing a response with an error while rolling back the write tests a different recovery problem.

The testing budget adds a second difficulty. A single failing rollout is cheap compared with a defensible reproducer: the baseline must pass, the perturbation must be valid, the final state must violate an independent requirement, and fresh runs must reproduce the relevant outcome. Searching for the largest number of raw failures can expend the budget on duplicate signatures or unstable effects. Conversely, aggressive confirmation can crowd out exploration. This motivates evaluating selection policies against the cost of completed evidence obligations rather than against failure yield alone.

Our original ambition was a new adaptive discovery algorithm. Source inspection narrowed that claim. Adaptive bandit fault injection already exists, as do counterfactual trace repair, coverage-guided agent testing, diversity scheduling and adaptive discovery with repeated sampling. We therefore present the implemented mechanism as an empirical index and test its incremental pooling hypothesis against confirmation-aware controls. A completed system does not resolve that novelty question.

This report makes three bounded contributions. First, it specifies and implements a reproducible stateful branch protocol with actual effects, delivered observations and independent final-state assertions. Second, it makes the cost of a fixed local confirmation obligation explicit and evaluates a simple predictive completion score at equal candidate access and operation budgets. Third, it releases raw outcomes, source manifests, replayable certificates, independent reviews and scripts that generate every comparative figure and table. The evidence concerns the supported workflows; it does not certify arbitrary autonomous agents.

2 Related Work and Novelty Boundary

Diagnosis, intervention and repair. AgentRx diagnoses existing failed traces using executable checks and grounded judgments [13]. Its inspected checkpoint manager tracks pipeline progress; that is different from restoring an external tool environment. CausalFlow already replaces trace steps and re-executes downstream work through a supplied executor [5]. Its source distinguishes executed branches from model-predicted outcomes when no executor is available. Neither trace branching nor minimal counterfactual repair is new here. Localization accuracy and new-failure discovery count are also different comparison targets.

Stateful agent evaluation. ToolSandbox provides state histories and complete execution-context copies [12]. AgentDojo executes tools over copied typed environments with task-specific utility and security checks [7]. ToolEmu instead expands scenario coverage using LM-emulated tools and LM-based risk evaluation [15]. These methods motivate separating executed state evidence from a plausible simulated outcome. Our integration reuses ToolSandbox's real offline reminder tools; it is one adapted workflow, not an official benchmark score. Source audits of τ_2-bench and BrowserGym further show that restoring a browser or replaying messages does not automatically restore all backend effects [16, 17].

Adaptive agent testing. The indexed method of an anonymous failure-forecasting submission already selects vulnerable steps in successful traces, uses a bandit over error categories and verifies perturbed continuations [2]. Full access was unavailable; we treat this as a serious overlap, not a replicated comparator. FLARE adjusts seed and mutation selection from behavioral coverage [9]. AgentLAB adaptively rewrites environmental injections based on execution feedback [11]. AFuzz uses diversity scheduling of bug mechanisms and scenario deduplication, although its target is software bugs found by agents rather than failures of the agents themselves [14]. Agent Error Simulator evaluates recovery using declarative format, logic and tool-call faults; its institutional record was inspected, but its experiments were not replicated [3].

Discovery, repetition and validity. Good-UCB optimizes distinct interesting-item discovery using missing-mass estimates [6]; STADS applies species-discovery reasoning to testing and discusses adaptive sampling bias [4]. Repeated adaptive sampling with false-discovery control predates this system [10]. A novelty/cost bandit score alone is therefore insufficient originality. AgentBoundary constructs matched executable families that separate apparent risk from action permissibility [18]. SSCBench distinguishes counterevidence that is available from evidence actually visible before reliance [8]. Metamorphic testing offers useful relation-based checks, but such relations do not replace an independent external-state contract [1].

The remaining empirical question is narrow: does a public-feature empirical prior improve the implemented completion index at the total budget and evidence rules used here? This study does not claim to solve the general optimal discovery problem or to outperform all these systems on their own tasks.

3 Problem and Evidence Contract

A task is $e = (x, s_0, \mathcal{T}, J)$: an instruction x, initial environment state s_0, supported tool transitions $\mathcal{T}$, and an independently specified outcome predicate J. The agent has resumable internal state h_t and a pending tool action a_t. We distinguish the actual transition and result,

$$(s_{t+1}, r_t) = T(s_t, a_t), \tag{1}$$

from the observation delivered to the agent, $o_t = D(r_t, s_t, a_t)$. A lost acknowledgment modifies D after T commits. A pre-effect timeout prevents the transition. A concurrent source change modifies an explicitly permitted part of s_t before the pending action.

An intervention i specifies its trigger, affected channel, permitted change and applicability conditions. It is admitted only when those conditions match the supported contract. A valid test need not preserve the original success outcome: permission loss can make safe refusal correct. The outcome predicate must encode that acceptable behavior before the result is observed.

For a matched baseline trajectory τ_0 and branch τ_i, a local failure witness requires

$$J(e, \tau_0) = 1, \quad V(e, i) = 1, \quad J(e_i, \tau_i) = 0. \tag{2}$$

The verifier reads exported database rows and actual file bytes. The agent's declaration is recorded separately. A silent failure is a violated postcondition accompanied by a parsed success assertion; a failure recognized by the agent remains a failure but is not silent.

Discovery units. The search reward uses a signature consisting of the violated requirement set and declared intervention kind. This is observable and reproducible. It is not a root-cause identity: two triggers can expose one defect, and one signature can hide several causes. Every numerical discovery result in this report uses requirement/intervention signatures. Independent mechanism counts would require additional evidence, such as controlled correction-response experiments.

Confirmation and minimality. A positive probe receives no reward. Three additional prespecified fresh seeded pairs must reproduce its signature. A negative

or invalid repeat closes the candidate. These four executions give a local repeated witness, not a population failure-rate bound or a family-wise statistical guarantee. Each perturbation is one atom in the supported language; removing it gives the identity control. Passing identity and failing the atom supports only 1-minimality within that language. We do not implement general compound-intervention or stochastic delta debugging.

4 Implementation and Independent Certificates

COUNTERGLASS supports three authored environment families. The ledger family stores jobs and checks exactly-one semantics, payload correctness and preservation of unrelated records. The inventory family reserves a complete product set through a paginated listing and checks exact coverage and source preservation. The artifact family transforms the current source file, checks version-sensitive output, and permits a safe stop after permission revocation. These are actual SQLite/file operations in temporary directories, rather than predicted state changes.

Before each tool call, the engine records a SQLite snapshot, file bytes, declared metadata, the waiting agent state and the pending action. A branch reconstructs the environment and checks its logical digest before executing that action. Supported scripted agents are finite-state JSON policies with naive retry behavior, reconciliation, complete guards, or a seeded mixture. Within a pair, the saved policy state is shared. Across confirmations, the policy is initialized with a fresh recorded seed. This exact coupling is suitable for these agents, not a universal stochastic-agent replay guarantee.

Interventions include identity, one pre-effect timeout, a lost acknowledgment after a committed write, a provenance-backed stale file read, a permitted listing-order change, a source update before writing, and permission loss. The trace preserves actual tool results and delivered observations separately. Untriggered or incompatible interventions are invalid, rather than evidence of robustness.

A certificate contains the canonical task, initial-state specification, branch checkpoint, intervention and validity explanation, baseline and branch traces, actual final states, independent assertions, recorded seeds, repetition outcomes, source fingerprints and reproduction commands. A separate checker imports no search code and recomputes the supported state/transition assertions. The portable checker enforces bounded policy invariants; it does not prove every saved internal-state transition. Source-matched fresh replay is required for the named-policy continuation claim. A fresh-process reproduction executes the installed agent and environment again; checking a saved trace and re-executing it are reported as separate operations. Hashes identify bytes; they are not signatures of authorship or a proof that the oracle matches a real user's intent.

Controls and corrections. Known-answer checks cover identity, deliberately defective retries, complete guards, incompatible/unreachable faults, permission loss, restored-state identity, malformed or semantically changed certificates and fresh replay. A correction is evaluated by changing the agent's guard or reconciliation behavior while holding the task and intervention fixed. This demonstrates a bounded engineering improvement when unchanged controls also pass; it does not establish an autonomous repair system.

5 Certificate-Completion Selection

Each candidate c has public features, a conservative per-pair cost bound d_c, its own observations, a status and possibly a witnessed signature. Outcomes are not available in advance. Candidates are grouped by agent identity, triggering tool and intervention kind, all public context. No authored defect label is exposed.

Let p_c denote the probability of a valid matching positive outcome under the selector's binary approximation. With beta parameters (a_c, b_c) and k remaining positive observations, the prior-predictive all-positive probability is

$$P_k = \mathbb{E}[p_c^k] = \prod_{j=0}^{k-1} \frac{a_c + j}{a_c + b_c + j}. \tag{3}$$

The expected number of trials until rejection or completion is

$$C_k = \sum_{j=0}^{k-1} \mathbb{E}[p_c^j]. \tag{4}$$

Both quantities follow directly by integrating a beta density and summing survival probabilities. They are standard calculations, not new theoretical results. The score is

$$I(c) = \frac{\nu_c P_k}{d_c C_k}, \tag{5}$$

where ν_c is a novelty heuristic. For a known signature, novelty is zero after its certification and one otherwise. For an untested candidate, novelty is discounted by the fraction of its bucket's observed positive events belonging to already certified signatures, with one unit of unseen mass. This heuristic does not have a Good–Turing confidence interpretation.

The independent variant starts from $\mathrm{Beta}(1, 1)$ and updates with the candidate's own outcomes. The pooled variant replaces that prior by one of strength two whose mean is the smoothed positive fraction among other observed bucket trials. The candidate's observations are excluded when forming that mean, then included in its own update. Reproduction evidence is never pooled: every rewarded candidate must finish its own obligation.

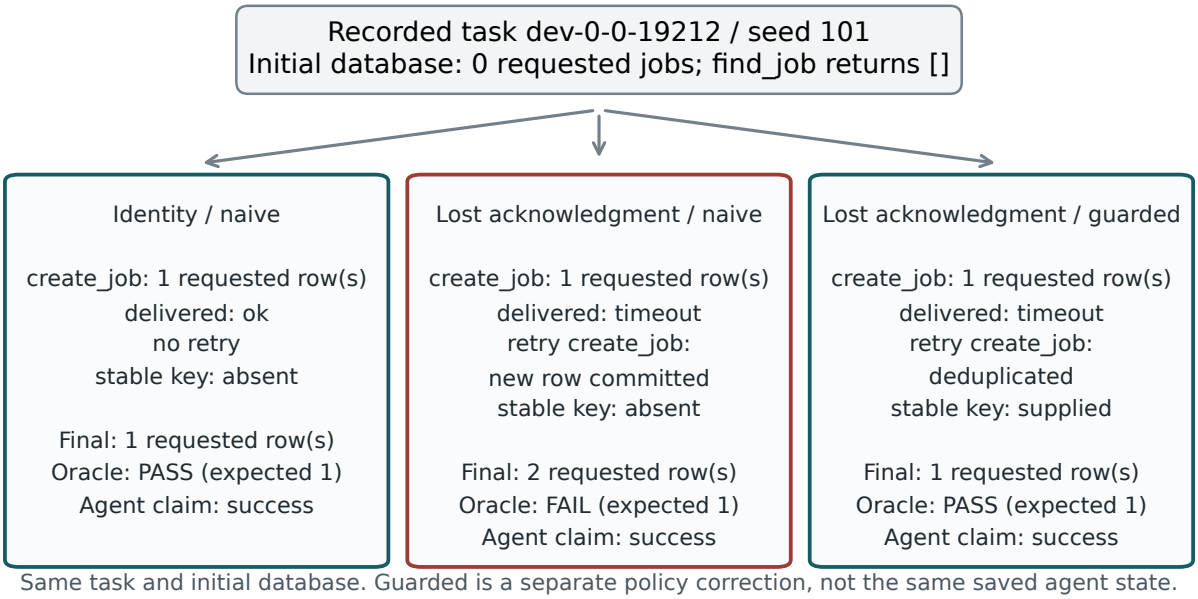


Figure 1: One recorded ledger case, generated directly from the saved certificate, identity control and corrected-policy execution. The first write commits in both faulted lanes; suppressing its response changes what the agent knows, not whether it happened. The naive retry creates a second row, while the guarded retry uses a stable idempotency key and is deduplicated. The corrected agent is a different controlled policy copy; the shared root denotes the task and initial database, not identical internal agent state. This is a case illustration, not aggregate telemetry.

Assumptions and limits. The pooled parameters are an empirical prior, not a calibrated hierarchical posterior. Adaptive confirmation gives successful candidates more observations; heterogeneous candidates need not form an iid beta likelihood. Moreover, the probe accepts any signature whereas repeats require the original signature. The binary approximation is exact only under a stable signature-specific observation model; in multi-signature environments it is a scoring approximation. No optimality, regret, causal-identification or false-discovery guarantee is asserted.

Obligations are complementary: an early repeat can have no completed reward while the last repeat finishes a certificate. Ordinary submodular-coverage guarantees cannot be transferred automatically. Conversely, treating confirmation blocks as atomic actions gives strong simple baselines. The implemented index does not solve a finite-horizon portfolio over validity, minimization and reproduction actions.

Operational procedure and cost. At every step, select the largest eligible score, run the actual matched pair, debit measured operations, check final state, update the candidate and bucket observations, and stop when no remaining complete obligation fits the bound. A new certificate updates the known-signature set. Other policies use the same acceptance rules and immediately service pending confirmations. Candidate/repetition seeds are derived from study seed, candidate identifier and repetition index, so policy ordering cannot change the coupled outcome.

Implemented completion-index loop

1. Initialize candidate evidence, bucket counts and certified signatures; charge common candidate generation.
2. Form the eligible set: untested/pending candidates with positive novelty whose complete remaining obligation fits the public bound.
3. If empty, stop. Otherwise choose the largest $I(c)$, using a pre-observation random tie order.
4. Execute a fresh matched baseline/branch pair with the candidate/repetition seed. Debit actual operations; reject a violated admission bound.
5. Independently verify baseline success, intervention validity and the branch requirement signature. A negative or changed signature closes the candidate.
6. Record a positive probe without reward. After three additional matching positives, award its signature only if new. Update bucket statistics and return to step 2.

For N candidates, maximum obligation length $r+1$, and L stored signature entries, a direct pass is $O(Nr + NL)$ in the worst case because novelty sums bucket counts. Status maintenance and pooled updates use linear candidate memory plus signature counts. Execution evidence and checkpoints are retained separately and can dominate storage. Selection CPU is measured rather

than assumed negligible. Optimizing this index's implementation is distinct from establishing its discovery value.

6 Evaluation Protocol

The versioned protocol predates the comparative pilot and held-out task generation. The completed locked input is `evidence/locked-v1`. The final study includes 1280 episodes: 32 seeds, four additional execution budgets (120, 240, 480, 960), and 10 policies. Each seed supplies six tasks, two per authored family, and naive, reconciliation and stochastic scripted configurations. Held-out identifiers and values differ from development, while workflow structures remain the same. This is not unseen-structure evaluation.

Policies include random valid candidates, feature coverage, fixed write/transport risk, schema order, failure-yield UCB-inspired selection, Good–Turing-inspired missing mass, novelty Thompson sampling, a fixed model-proposed order, and the independent and pooled completion indices. Published-inspired criteria are adaptations, not faithful reproductions of external methods. They share candidate access, public metadata, independent verifier, repetition threshold and budget semantics. The independent completion policy is the prespecified confirmation-aware primary comparator. The model-order baseline uses one recorded ordering of public fault types, reused across episodes; its ranking does not adapt to their outcomes and it does not synthesize new interventions. The proposal is charged unamortized to each standalone baseline episode: 552 tokens and 43.107 seconds. This accounting is a conservative per-episode charge, not a claim that the shared proposal was actually regenerated for each episode.

All-in accounting. Each selected pair pays the baseline and branch suffix's agent-facing environment operations, two independent outcome checks and one state restoration. Conservative admission reserves $2n + 8$ per future pair, where n is the task's baseline operation count; actual cost is debited. Candidate generation executes baselines and its measured startup charge is added to every standalone policy cost. An additional baseline pass used to determine public bounds is charged too. Selection CPU, actual episode wall time, checkpoint counts, serialization bytes, agent steps and model calls/tokens are recorded separately. These operation units are not money, token cost or a universal computational metric. The worker-process peak resident-memory measurement is cumulative across preceding episodes; it is not an isolated per-policy memory cost.

Primary outcome and uncertainty. The primary outcome is paired completed-signature count at additional budget 480, with identical candidate-generation operation cost within each seed. Mean signatures divided by realized full cost is supporting information, not a replacement estimand. The frozen improvement target requires at least 20% higher mean count than independent completion, a primary paired interval lower endpoint above zero, no mean loss against independent completion at any other tested budget, and a larger primary mean than every other baseline. This composite target is set after the pilot and before held-out task generation. We use 10,000 deterministic paired bootstrap resamples of seeds and report percentile 95% descriptive intervals. The resampling unit is an authored task seed, not an independent application or failure mechanism. Other budgets and pairwise comparisons are supporting analyses, not additional significance tests.

Raw unsuccessful searches, negative repeats, invalid interventions and unused budgets remain in the data. An interrupted first pilot is preserved rather than mixed into final results. The pilot informed runtime feasibility; source fingerprints and the final lock distinguish it from the held-out evaluation. No optimizer tuning against final outcomes is justified by a negative result.

7 Results

Primary comparison. The pooled and independent indices achieve 4.031 and 3.812 mean signatures, respectively, at the primary cap. The paired difference is +0.219 [+0.094, +0.375], with 7 seed wins, 25 ties and 0 losses. The observed relative gain is 5.74%, below the frozen 20% requirement; schema order reaches 4.719 signatures. The frozen composite improvement target is not met. This is a negative result for the tested selection hypothesis at the prespecified criterion. The complete comparison is in Table 1; Table 4 retains all supporting paired differences.

Budget, cost and ablation. Figure 2 reports every operating point; Figure 3 includes realized startup and execution cost. Figure 4 isolates removal of the pooled empirical prior without changing completion obligations. At the primary budget, the pooled policy uses 643.06 mean full operation units, 1.729 seconds of mean standalone wall time and 0.1124 seconds of mean selection time. Operation-count advantages must be assessed alongside these actual timings. The manifest records 4 seed workers; fixed within-seed policy order, concurrent host load and the unamortized model-proposal charge make wall times descriptive rather than clean hardware-normalized speed comparisons. The maximum recorded worker-process resident-memory peak is 36.0 MiB, a cumulative high-water mark rather than a policy-specific measurement.

Evidence completion and diversity. Table 2 reports selected valid pairs, probe hits, and matching con-

Policy	Signatures	Full cost	/100 cost	Unused	Wall (s)	Select (s)
Random + confirmation	2.719	648.8	0.420	38.3	1.486	0.0223
Feature coverage	3.594	648.9	0.555	38.2	1.530	0.0495
Fixed risk	2.344	643.8	0.365	43.4	1.410	0.0448
Schema order	4.719	648.4	0.729	38.8	1.535	0.0712
Fixed model-proposed order	2.938	644.7	0.456	42.5	44.553	0.0479
Yield-UCB inspired	4.000	642.8	0.623	44.3	1.816	0.1020
Good-Turing inspired	4.000	643.4	0.622	43.8	1.854	0.1114
Novelty Thompson	3.938	643.0	0.613	44.2	1.805	0.1053
Completion, independent	3.812	644.2	0.592	43.0	1.820	0.1079
Completion, pooled	4.031	643.1	0.628	44.1	1.729	0.1124

Table 1: Primary budget 480. Means across held-out seeds. Full cost includes common candidate generation; "/100 cost" is the mean per-seed realized-cost ratio. Wall time includes startup, actual search and the unamortized model-proposal charge where applicable; selection time is included within search and shown separately. These are different cost measures.

firmation outcomes. The confirmation fraction is conditional on adaptive selection; it is not an unbiased estimate of deployment reliability. Repeated silent-failure events count repeated tests, not independent defects. Figure 5 shows which signatures were completed across seeds, exposing concentration on easy signatures rather than treating raw failing-trace count as diversity.

Negative evidence and interpretation. The observed coverage explains what the headline count leaves out. At the primary cap, pooled completion certifies the inventory lost-ack signature in 1/32 seeds and listing-order signature in 0/32, versus 30/32 and 27/32 for schema order. Its completions instead concentrate on the short artifact and ledger workflows. This pattern is consistent with a cost-normalized index favoring short branches, but does not isolate that explanation: family, cost, intervention semantics and novelty are confounded. No policy is retuned from this diagnostic. Mean selection time is 0.1124 seconds for pooling, versus 0.1079 for independent completion and 0.0712 for schema order. The descriptive primary improvement over independent completion therefore does not establish a useful overall search advantage. The system's functioning branching and evidence pipeline remain supported independently of this negative finish-line result.

8 Separate Execution Case Studies

Actual ToolSandbox tools. A pinned upstream ToolSandbox subset executes an adapted reminder workflow through its real offline context and reminder functions. Three conditions each have five paired executions: naive agent with lost acknowledgment, guarded agent with the same fault, and naive agent with identity. All 15 baselines satisfy the contract. The five naive faulted continuations create duplicates and falsely claim success; the five guarded and five identity continuations satisfy the independent reminder-state assertions. The unrelated reminder and other sandbox tables are checked for preservation.

This is one scripted duplicate-write mechanism, not 15 distinct discoveries or an official ToolSandbox benchmark result. Fresh UUIDs and timestamps vary, so semantic assertions are reproduced rather than byte-identical states. A second complete recording reproduces the same counts. A portable checker independently validates all 30 recorded pairs without importing ToolSandbox or trusting its outcome flags; a separate fresh upstream replay also passes. Upstream license, revision and dependency evidence are retained. The adapter demonstrates that the effect/observation distinction transfers to actual public tools; it does not evaluate the adaptive selector on public benchmark tasks.

Local model. A separate bounded case study uses an already installed custom model reported by Ollama as a Qwen-family architecture with 35.5B parameters and Q3_K_M quantization through a local server. The artifact records the model digest, options, outputs and observed usage. The base-weight provenance is not independently established. Two requested seeds are attempted for each of standard and reconciliation instructions, on one exactly-once job task with a lost acknowledgment.

One standard baseline times out before a valid pair is obtained. The other standard baseline succeeds; its branch queries the committed job, then creates another with a new idempotency key. The independent database oracle observes two jobs. Its eventual output recognizes the problem and is not a parsed success assertion, so this is an actual state failure, not a silent false-success example. Both reconciliation-instruction pairs complete their original and faulted tasks.

The first adapter version counted only completed model responses and recorded the timed-out request as zero calls. Preserved accounting notes correct this to one attempted request and 60.44 seconds of client time with unknown server token/compute cost. The successful re-

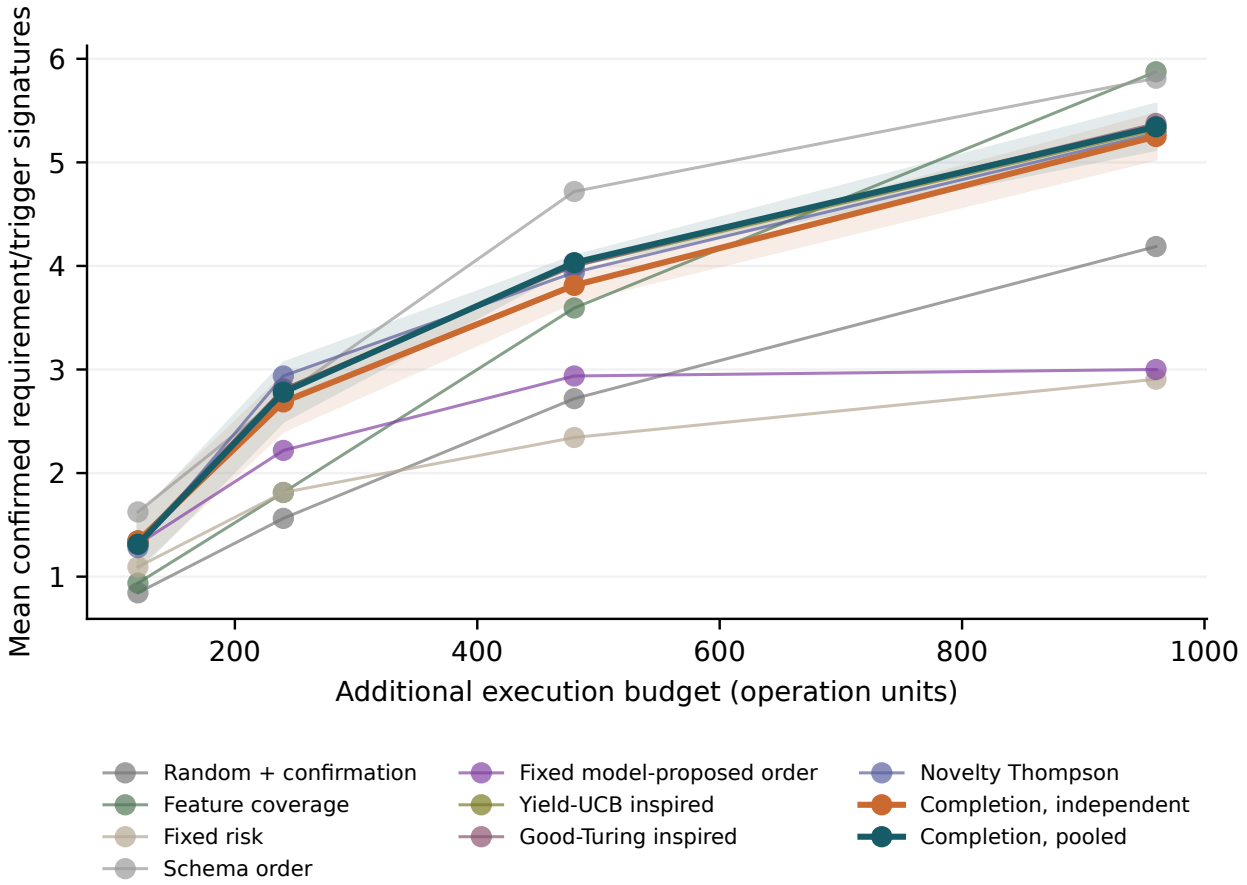


Figure 2: Recorded discovery at all prespecified additional execution budgets. Each point comes from a separate complete budgeted episode; budget-dependent admission can change its search path, so these are not prefixes of one trajectory. Shaded bands are descriptive paired-seed bootstrap intervals for the completion variants. Common candidate-generation cost is included separately in Table 1 and Figure 3. The ordinate counts requirement/trigger signatures, not root causes.

sponses report 19 completed calls and 19,140 tokens; including the timeout gives 20 attempted calls and a token lower bound. These few cases do not establish a failure rate, superiority of an instruction policy, or selector transfer. Temperature zero and saved seeds do not guarantee exact model outputs.

9 Independent Review and Threats to Validity

An independent implementation/raw-data review checked the first 72 complete pilot episodes, including cost sums, fixed confirmation thresholds and 2,735 policy-shared trial identities. It found no invalidating defect for the narrow comparison and identified five presentation/modeling issues: ambiguous count-versus-ratio wording, empirical-prior calibration, changing initial/repeat event definitions, published-inspired baseline fidelity, and signature inflation relative to root causes. The protocol clarified the primary estimand before final lock; the other issues remain explicit limits rather than being concealed by stronger terminology.

Complete post-lock outcome audit. After the search and target were frozen, a separate verifier independently reconstructed all 7,177 unique candidate/repetition pairs represented by the 54,619 recorded trial occurrences. It loaded the frozen engine bytes, re-executed each saved policy seed, recomputed state transitions and contracts through the standalone checker, and compared every deterministic cost, outcome assertion, signature and final-state hash. All 7,177 pairs matched, with zero discrepancies. Complete paired states and traces are retained as 32 compressed streams, including recovered and nonfailing outcomes. The audit consumed 136.44 seconds on four workers and is reported separately from discovery cost. It validates this recorded experiment; it neither reruns selection nor improves the negative discovery result.

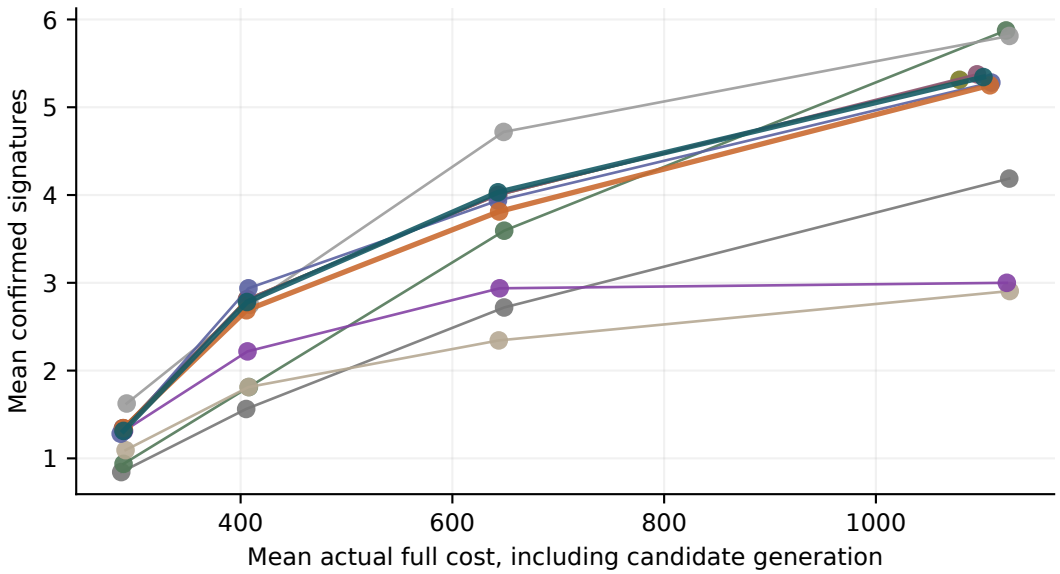


Figure 3: Recorded cost–quality tradeoff using mean actual full cost, including candidate generation. Colors match Figure 2. Points arise from complete budgeted episodes, not optimized frontiers.

Policy	Valid/selected	Probe hits/probes	Matched repeats/repeats	Silent events
Random + confirmation	1042/1042	139/732	281/310	420
Feature coverage	1153/1153	161/756	367/397	528
Fixed risk	1027/1027	198/755	244/272	442
Schema order	1162/1162	340/636	482/526	822
Fixed model-proposed order	1063/1063	356/727	302/336	658
Yield-UCB inspired	2047/2047	358/1622	399/425	757
Good-Turing inspired	2048/2048	296/1623	399/425	695
Novelty Thompson	2049/2049	275/1646	385/403	660
Completion, independent	2052/2052	257/1636	383/416	640
Completion, pooled	2042/2042	367/1600	407/442	774

Table 2: Raw selected-event denominators at the primary budget. Probe hits require valid baseline-pass/counterfactual-fail outcomes. A matching repeat also requires the original signature. These counts include repeated trials and policy overlap; they are not independent application samples.

Instructions	Seed	Baseline	Branch
Standard	1701	timeout	not admitted
Standard	1702	pass	fail
Reconciliation	1701	pass	pass
Reconciliation	1702	pass	pass

Table 3: All requested local-model case attempts, including the failed baseline. This table is a case study, not the comparative search experiment.

Oracle and validity risk. The same author supplies the synthetic requirements and agents. Independent checker code reduces reliance on the search implementation but can still share a misunderstood specification. Guarded agents and tampering controls are necessary, not exhaustive oracle validation. Fault admission is defined for a small declared surface, not for arbitrary compatible-looking API failures. The reference's ability to recover does not alone prove environmental realism.

Attribution and nondeterminism. Matched restoration supports a local effect claim under the controlled transition and policy-state assumptions. It does not establish a universal root cause. A first changed observation need not be the first consequential state divergence. The local-model case has additional server nondeterminism and no population causal estimate. Common seeds can reduce variation only under the actual coupling maintained by the adapter.

Selection and statistics. The empirical prior, signature taxonomy and cost units are design choices. Observed confirmations are selected; repeated success is not a calibrated confidence statement. Descriptive seed bootstrap intervals do not cover uncertainty over indepen-

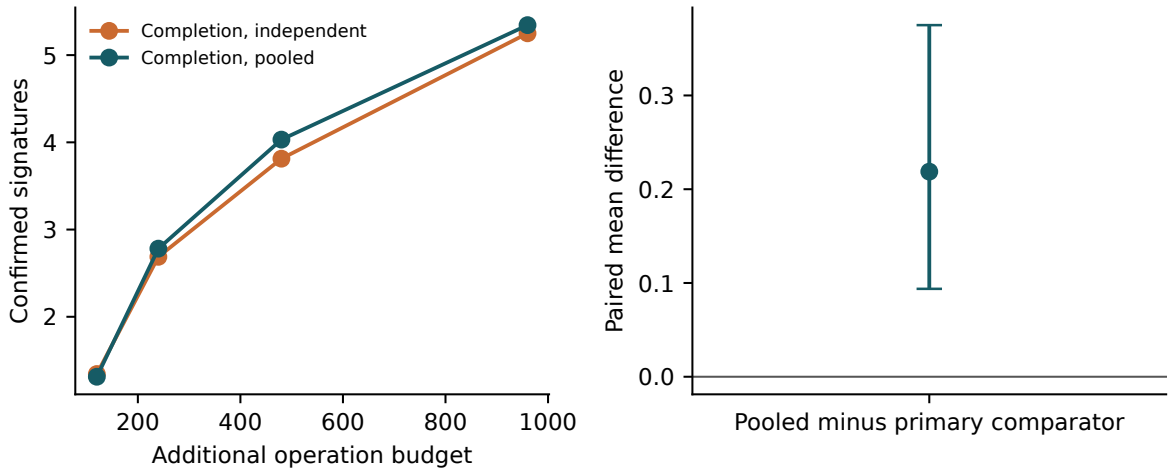


Figure 4: No-pooling ablation and the primary paired count difference. The interval reflects seed variation within the authored workflow structures.

dently sampled agent applications. Multiple supporting comparisons are not confirmatory hypothesis tests. Fixed serial policy order, shared-host effects and snapshot implementation affect wall time. Incremental per-policy memory and total timeout token consumption remain unknown.

Generalization and external comparison. Scripted agents are valuable known-answer controls but cannot establish LLM-agent discovery performance. The held-out split varies parameters rather than workflow structures. The ToolSandbox and model cases do not run the selector and cannot rescue a negative comparative result. Stronger future work would use unseen public workflow structures, two independent model/agent implementations, an audited common candidate surface and confirmation-aware external comparators at matched all-in cost.

10 Conclusion

COUNTERGLASS provides a functioning way to restore supported stateful workflows, intervene on actual execution or delivered observations, check postconditions and preserve a reproducible failure witness. The budgeted study evaluates a concrete empirical certificate-completion index and exposes its assumptions and costs. The frozen composite improvement target is not met. This is a negative result for the tested selection hypothesis at the prespecified criterion. The strongest supported contribution is inspectable engineering and bounded empirical evidence. The study does not establish performance on unseen workflow structures or a generally superior adaptive discovery algorithm.

Artifact and Reproduction

The repository contains a dependency-free Python core, CLI, independent checker, raw JSONL episodes, candidate manifests, protocol/lock, research audits, recorded external cases and this editable manuscript. With the separately documented plotting environment:

```
python scripts/analyze_results.py FINAL_RUN \
  --output paper/generated-final-v4 \
  --figures paper/figures-v4 --lock LOCK.json
```

The analyzer rejects incomplete studies, checks paired seed/outcome identity and certificate thresholds, and records input/analyzer hashes. It derives the tables and figures from raw episodes without importing the tested agents. Existing generated outputs are not silently overwritten. Certificate checking recomputes saved evidence; the separate replay command executes fresh supported trials. No external service, real account fault injection or publication is required.

References

[1] Metamorphic testing of multi-agent LLM systems: A trace-based behavioral oracle framework. 2026 IEEE International Conference on Artificial Intelligence Testing (AITest), 2026. URL https://ieeexplore.ieee.org/abstract/document/11662476/. Publisher metadata and abstract verified; full methods not audited.

[2] Anonymous Authors. Efficient forecasting of task failures in LLM agents through adaptive fault injection, 2026. URL https://openreview.net/pdf/f81b4d72b4ac58cbc7ec781bdc3f8e3c705d4650.pdf. Anonymous OpenReview submission; inspected indexed primary-PDF excerpts only; publication status unverified.

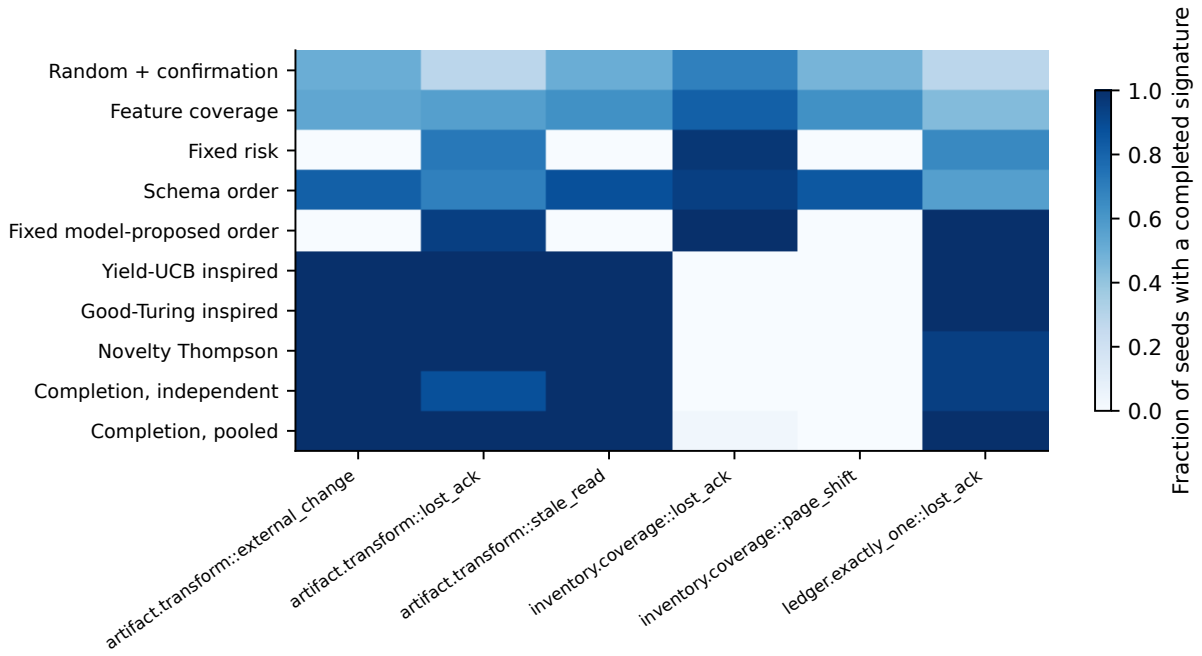


Figure 5: Fraction of seeds in which each policy completed each observed requirement/intervention signature at the primary budget. Signature definitions are fixed by requirements and trigger kinds; their number does not establish mechanism diversity.

[3] K. Bateman, J. Cernuda, L. Logan, B. Nicolae, F. Cappello, X.-H. Sun, and A. Kougkas. Agent error simulator: Fault injection for testing recovery in agentic workflows. In *22nd IEEE International Conference on eScience*, 2026. URL https://grc.iit.edu/gnosis/publications/bateman-2026-agent-error-simulator-3c7d/. Primary institutional metadata and abstract inspected; external results not replicated.

[4] Marcel Böhme. STADS: Software testing as species discovery. *ACM Transactions on Software Engineering and Methodology*, 2018. URL https://mboehme.github.io/paper/TOSEM18.pdf.

[5] Akash Bonagiri, Devang Borkar, Gerard Janno Anderias, Setareh Rafatirad, and Houman Homayoun. CausalFlow: Causal attribution and counterfactual repair for LLM agent failures. *arXiv preprint arXiv:2605.25338*, 2026. URL https://arxiv.org/abs/2605.25338.

[6] Sébastien Bubeck, Damien Ernst, and Aurélien Garivier. Optimal discovery with probabilistic expert advice: Finite time analysis and macroscopic optimality. *Journal of Machine Learning Research*, 14:601–623, 2013. URL https://www.jmlr.org/papers/v14/bubeck13a.html.

[7] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *arXiv preprint arXiv:2406.13352*, 2024. URL https://arxiv.org/abs/2406.13352.

[8] Xincheng He, Wanli Dong, Zhaoqiang Guo, Yan Liu, and Lei Xu. SSCBench: Evaluating the evidential validity of fault-injection tests for tool-using LLM agents. *arXiv preprint arXiv:2610.11514*, 2026. URL https://arxiv.org/abs/2610.11514.

[9] Mingxuan Hui, Xinyue Li, Lu Wang, Chengcheng Wan, Yifan Wang, Yimian Wang, Feiyue Song, Beining Shi, Yixi Li, and Yaxiao Li. FLARE: Agentic coverage-guided fuzzing for LLM-based multi-agent systems. *arXiv preprint arXiv:2604.05289*, 2026. URL https://arxiv.org/abs/2604.05289.

[10] Kevin Jamieson and Lalit Jain. A bandit approach to multiple testing with false discovery control. In *Advances in Neural Information Processing Systems*, 2018. URL https://arxiv.org/abs/1809.02235.

[11] Tanqiu Jiang, Yuhui Wang, Jiacheng Liang, and Ting Wang. AgentLAB: Benchmarking LLM agents against long-horizon attacks. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 53162–53180, 2026. URL https://proceedings.mlr.press/v306/jiang26as.html.

[12] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen

Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. *arXiv preprint arXiv:2408.04682*, 2024. URL https://arxiv.org/abs/2408.04682.

[13] Microsoft. AgentRx: Automated agent failure diagnosis, source repository, 2026. URL https://github.com/microsoft/AgentRx. Inspected revision 7a18c79708e7671be15124460f4f7296107c2a55.

[14] Junyoung Park and Insu Yun. Agentic fuzzing: Opportunities and challenges. *arXiv preprint arXiv:2605.10074*, 2026. URL https://arxiv.org/abs/2605.10074.

[15] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. *arXiv preprint arXiv:2309.15817*, 2023. URL https://arxiv.org/abs/2309.15817.

[16] ServiceNow. BrowserGym source repository, 2026. URL https://github.com/ServiceNow/BrowserGym. Inspected revision 27a90ce815098f43414e93491e06907af468e45b.

[17] Sierra Research. tau2-bench source repository, 2026. URL https://github.com/sierra-research/tau2-bench. Inspected revision da0835f6c67baef58bdda18f2a3b037dfb3a1754.

[18] Tianzhuo Yang, Zirui Mi, Yantao Huang, Guoxi Zhang, Jiawei Chen, Yaodong Yang, and Jingwei Yi. AgentBoundary: Counterfactual evaluation of safety in tool-using LLM agents. *arXiv preprint arXiv:2609.33658*, 2026. URL https://arxiv.org/abs/2609.33658.

A All Supporting Paired Comparisons

Comparator	Pooled minus comparator	95% paired interval	Win/tie/loss
Random + confirmation	+1.312	[+0.938, +1.656]	24/7/1
Feature coverage	+0.438	[+0.062, +0.812]	15/11/6
Fixed risk	+1.688	[+1.469, +1.906]	32/0/0
Schema order	-0.688	[-1.000, -0.375]	2/11/19
Fixed model-proposed order	+1.094	[+1.000, +1.219]	32/0/0
Yield-UCB inspired	+0.031	[+0.000, +0.094]	1/31/0
Good-Turing inspired	+0.031	[+0.000, +0.094]	1/31/0
Novelty Thompson	+0.094	[+0.000, +0.219]	3/29/0
Completion, independent	+0.219	[+0.094, +0.375]	7/25/0

Table 4: Pooled-minus-comparator signature counts at the primary cap. All intervals use the same paired seed resamples. Only the prespecified primary comparator is confirmatory in design; no multiplicity-corrected discovery claim is made.

Policy	Normalized budget AUC	95% seed interval
Random + confirmation	2.465	[2.270, 2.652]
Feature coverage	3.273	[3.047, 3.498]
Fixed risk	2.082	[1.936, 2.230]
Schema order	3.941	[3.738, 4.135]
Fixed model-proposed order	2.432	[2.348, 2.506]
Yield-UCB inspired	3.523	[3.436, 3.611]
Good-Turing inspired	3.539	[3.455, 3.623]
Novelty Thompson	3.508	[3.430, 3.584]
Completion, independent	3.414	[3.311, 3.521]
Completion, pooled	3.533	[3.441, 3.629]

Table 5: Supporting normalized trapezoidal area under signature count versus additional allowed budget, including the origin. Each budget is a separate episode, so this summarizes a budget-response curve, not one cumulative execution.

B Reading a Failure Certificate

A reviewer should inspect the requirement and initial state before looking at the agent's final text. Confirm that the branch snapshot matches the baseline boundary and that the fault's declared semantics preserve the intended actual effect. For a lost acknowledgment, the committed row must already exist before retry. Then recompute final-state assertions and compare the identity control. Finally inspect every repeated outcome, failed attempt and cost. A consistent hash and a persuasive explanation do not replace these checks.

For the exactly-once case, the decisive divergence is a second committed row for the same logical request. The delivered timeout explains why retry was considered, but the actual state and request identity establish the violation. Preserving an idempotency key or reconciling authoritative state is a bounded correction. Its success on this case does not prove that all retry failures or future applications are repaired.